

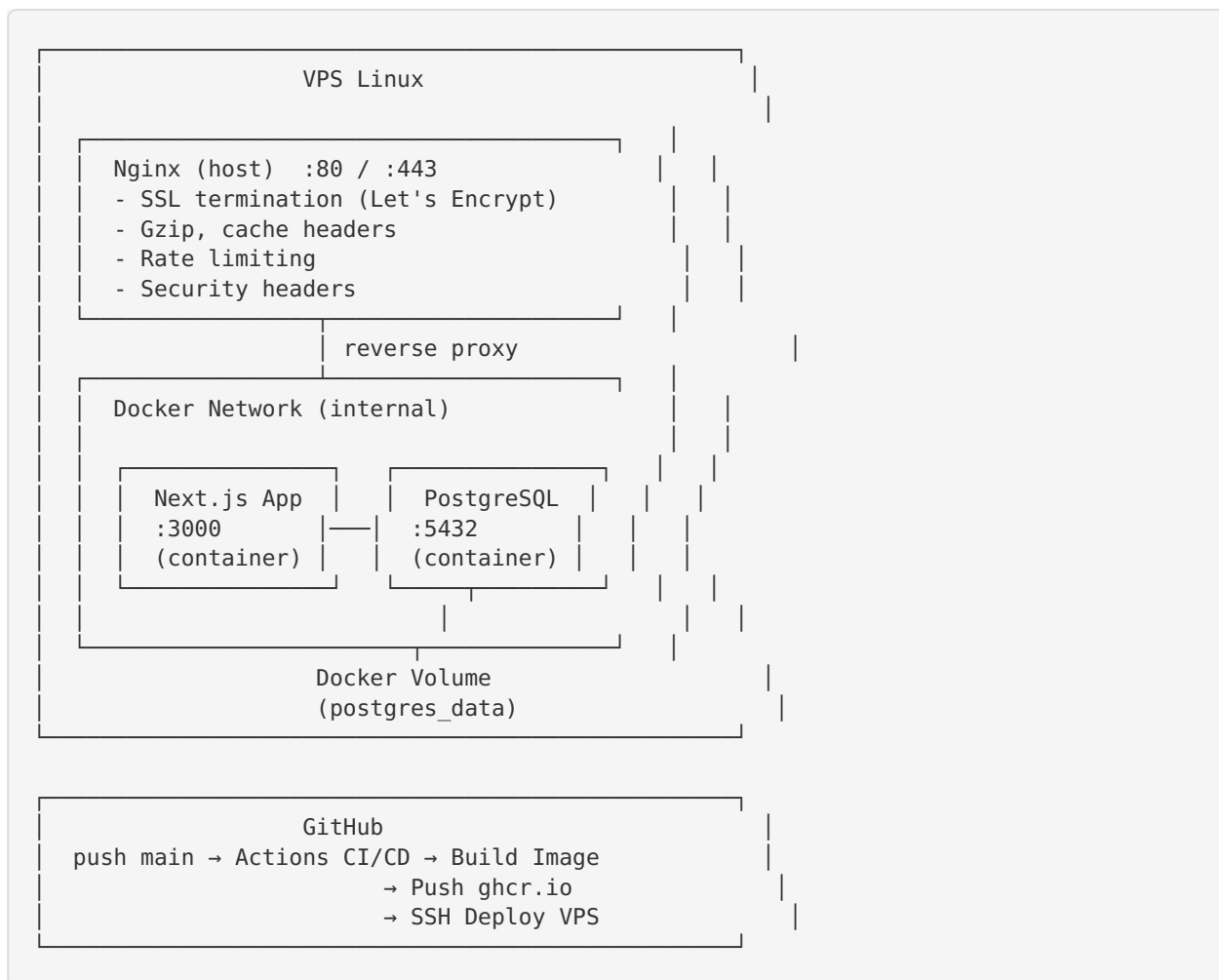
Hướng Dẫn Deploy Production - Reiseblog

Tài liệu này hướng dẫn deploy Next.js travel blog lên VPS Linux với Docker, Nginx, SSL và GitHub Actions CI/CD.

Mục Lục

1. [Kiến Trúc Tổng Quan](#)
 2. [Chuẩn Bị VPS](#)
 3. [Cài Đặt Phần Mềm](#)
 4. [Cấu Hình GitHub Secrets](#)
 5. [Deploy Lần Đầu](#)
 6. [Cấu Hình SSL](#)
 7. [Security Hardening](#)
 8. [Redeploy Khi Push Code Mới](#)
 9. [Rollback](#)
 10. [Monitoring & Logs](#)
 11. [Database Migrations](#)
 12. [Troubleshooting](#)
-

1. Kiến Trúc Tổng Quan



Quyết định kỹ thuật:

- **Registry:** GitHub Container Registry (ghcr.io) - miễn phí, tích hợp sẵn với GitHub Actions
- **Nginx trên host:** Để cấu hình SSL, rate limiting, không cần container thêm
- **PostgreSQL trong Docker:** Dễ backup, dễ migrate, volume persistent
- **Standalone output:** Next.js build nhỏ gọn, không cần full node_modules

2. Chuẩn Bị VPS

Yêu cầu tối thiểu

- **OS:** Ubuntu 22.04 LTS hoặc 24.04 LTS
- **RAM:** 1GB+ (khuyến nghị 2GB)
- **Disk:** 20GB+ SSD
- **CPU:** 1 vCPU+
- **IP:** có IP public
- **Domain:** đã trỏ A record về IP của VPS

Bước 1: Truy cập VPS

```
ssh root@your-vps-ip
```

Bước 2: Tạo deploy user (không dùng root)

```
# Tạo user
adduser deploy
usermod -aG sudo deploy

# Copy SSH key
mkdir -p /home/deploy/.ssh
cp ~/.ssh/authorized_keys /home/deploy/.ssh/
chown -R deploy:deploy /home/deploy/.ssh
chmod 700 /home/deploy/.ssh
chmod 600 /home/deploy/.ssh/authorized_keys

# Test login từ máy local:
# ssh deploy@your-vps-ip
```

Bước 3: Update system

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl wget git unzip htop
```

3. Cài Đặt Phần Mềm

3.1 Docker & Docker Compose

```
# Cài Docker
curl -fsSL https://get.docker.com | sh

# Thêm user deploy vào group docker
sudo usermod -aG docker deploy

# Verify (logout/login lại nếu cần)
docker --version
docker compose version
```

3.2 Nginx

```
sudo apt install -y nginx
sudo systemctl enable nginx
sudo systemctl start nginx

# Verify
nginx -v
curl -I http://localhost
```

3.3 Certbot (Let's Encrypt)

```
sudo apt install -y certbot python3-certbot-nginx
certbot --version
```

3.4 Tạo thư mục ứng dụng

```
# Login với user deploy
su - deploy

# Tạo thư mục
sudo mkdir -p /opt/reiseblog
sudo chown deploy:deploy /opt/reiseblog

# Tạo thư mục cho Certbot
sudo mkdir -p /var/www/certbot
```

4. Cấu Hình GitHub Secrets

Vào **GitHub repo** → **Settings** → **Secrets and variables** → **Actions**, thêm các secrets sau:

Secret Name	Mô tả	Ví dụ
SSH_HOST	IP hoặc domain của VPS	123.456.789.0
SSH_USER	User SSH (không dùng root)	deploy
SSH_PORT	SSH port	22 (hoặc port custom)
SSH_PRIVATE_KEY	Nội dung private key SSH	Nội dung file ~/.ssh/id_ed25519

Tạo SSH key cho GitHub Actions:

```
# Trên máy local:
ssh-keygen -t ed25519 -C "github-actions-deploy" -f ~/.ssh/deploy_key

# Copy public key lên VPS:
ssh-copy-id -i ~/.ssh/deploy_key.pub deploy@your-vps-ip

# Lấy private key để paste vào GitHub Secret SSH_PRIVATE_KEY:
cat ~/.ssh/deploy_key
```

Login Docker Registry trên VPS:

```
# Trên VPS (user deploy):
# Tạo GitHub Personal Access Token (PAT) với quyền read:packages
# Vào: GitHub → Settings → Developer settings → Personal access tokens

echo "YOUR_GITHUB_PAT" | docker login ghcr.io -u YOUR_GITHUB_USERNAME --password-stdin
```

5. Deploy Lần Đầu

5.1 Upload files lên VPS

```
# Từ máy local, copy các file cấu hình lên VPS:
scp docker-compose.production.yml deploy@your-vps-ip:/opt/reiseblog/
scp .env.production.example deploy@your-vps-ip:/opt/reiseblog/
scp -r deploy/scripts deploy@your-vps-ip:/opt/reiseblog/
```

5.2 Cấu hình environment

```
# Trên VPS:
cd /opt/reiseblog
cp .env.production.example .env.production
nano .env.production
```

Sửa các giá trị thật:

```
# Database
POSTGRES_DB=reiseblog
POSTGRES_USER=reiseblog
POSTGRES_PASSWORD=your_strong_password_here

# Auth
NEXTAUTH_URL=https://yourdomain.com
NEXTAUTH_SECRET=$(openssl rand -base64 32)

# Docker
DOCKER_IMAGE=ghcr.io/your-username/reiseblog:latest

# S3 (nếu có)
AWS_ACCESS_KEY_ID=xxx
AWS_SECRET_ACCESS_KEY=xxx
AWS_REGION=eu-central-1
AWS_BUCKET_NAME=xxx
AWS_FOLDER_PREFIX=xxx

# Migration lần đầu
RUN_MIGRATIONS=true
```

Generate NEXTAUTH_SECRET:

```
openssl rand -base64 32
```

5.3 Push code + build lần đầu

Push code lên branch `main` để GitHub Actions tự build và deploy.

Hoặc build thủ công lần đầu:

```
# Trên VPS, build local (nếu chưa có image):
cd /opt/reiseblog
git clone https://github.com/your-username/reiseblog.git src
cd src/nextjs_space
docker build -t ghcr.io/your-username/reiseblog:latest .
cd /opt/reiseblog
```

5.4 Khởi chạy

```
cd /opt/reiseblog

# Chạy với migration lần đầu:
RUN_MIGRATIONS=true docker compose -f docker-compose.production.yml \
  --env-file .env.production up -d

# Kiểm tra:
docker compose -f docker-compose.production.yml ps
docker logs -f reiseblog-app
```

5.5 Tạo admin user đầu tiên

Sau khi migration xong, tạo admin user:

```
# Truy cập PostgreSQL container:
docker exec -it reiseblog-db psql -U reiseblog -d reiseblog

# Hoặc dùng app để seed data (xem scripts/seed.ts nếu có)
```

6. Cấu Hình SSL

Bước 1: Cấu hình Nginx ban đầu (HTTP only)

```
# Copy config HTTP tạm:
sudo cp /opt/reiseblog/deploy/nginx/nginx-initial-http.conf \
  /etc/nginx/sites-available/reiseblog

# Sửa domain trong file:
sudo nano /etc/nginx/sites-available/reiseblog
# Thay 'yourdomain.com' bằng domain thật

# Enable site:
sudo ln -sf /etc/nginx/sites-available/reiseblog /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default

# Test + reload:
sudo nginx -t
sudo systemctl reload nginx
```

Bước 2: Cấp SSL với Certbot

```
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com

# Hoặc dùng standalone mode:
sudo certbot certonly --webroot -w /var/www/certbot \
-d yourdomain.com -d www.yourdomain.com
```

Bước 3: Chuyển sang config HTTPS đầy đủ

```
# Copy config HTTPS đầy đủ:
sudo cp /opt/reiseblog/deploy/nginx/reiseblog.conf \
/etc/nginx/sites-available/reiseblog

# Sửa domain:
sudo nano /etc/nginx/sites-available/reiseblog
# Thay TẤT CẢ 'yourdomain.com' bằng domain thật

# Test + reload:
sudo nginx -t
sudo systemctl reload nginx
```

Bước 4: Auto Renew SSL

```
# Test renewal:
sudo certbot renew --dry-run

# Certbot tự thêm cron job. Kiểm tra:
sudo systemctl list-timers | grep certbot

# Hoặc thêm thủ công vào crontab:
sudo crontab -e
# Thêm dòng:
0 3 * * * certbot renew --quiet --deploy-hook "systemctl reload nginx"
```

7. Security Hardening

7.1 UFW Firewall

```
sudo ufw default deny incoming
sudo ufw default allow outgoing

# Chỉ mở các port cần thiết:
sudo ufw allow ssh          # Port 22 (hoặc port custom)
sudo ufw allow 80/tcp       # HTTP (cần cho Certbot + redirect)
sudo ufw allow 443/tcp      # HTTPS

# Bật:
sudo ufw enable
sudo ufw status verbose
```

Lưu ý: KHÔNG mở port 5432 (PostgreSQL) và 3000 (Next.js) ra public.
 PostgreSQL chỉ được truy cập qua Docker internal network.
 Next.js chỉ listen trên 127.0.0.1:3000.

7.2 Fail2Ban

```
sudo apt install -y fail2ban

# Tạo config:
sudo tee /etc/fail2ban/jail.local << 'EOF'
[DEFAULT]
bantime = 3600
findtime = 600
maxretry = 5

[sshd]
enabled = true
port = ssh
filter = sshd
logpath = /var/log/auth.log
maxretry = 3
bantime = 7200

[nginx-http-auth]
enabled = true

[nginx-limit-req]
enabled = true
filter = nginx-limit-req
logpath = /var/log/nginx/reiseblog-error.log
maxretry = 10
bantime = 600
EOF

sudo systemctl enable fail2ban
sudo systemctl restart fail2ban
sudo fail2ban-client status
```

7.3 SSH Hardening (optional)

```
sudo nano /etc/ssh/sshd_config
```

Các thay đổi khuyến nghị:

```
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
Port 2222                # Đổi port SSH (optional)
```

```
sudo systemctl restart sshd

# Nếu đổi port, cập nhật UFW:
sudo ufw allow 2222/tcp
sudo ufw delete allow ssh
```


QUAN TRỌNG: Test SSH với port mới TRƯỚC khi đóng session hiện tại!

7.4 Docker Security

Đã được cấu hình trong Dockerfile và docker-compose:

- App chạy với user `nextjs` (non-root, UID 1001)
- Resource limits: 512MB RAM, 1 CPU
- PostgreSQL không expose port ra ngoài
- Internal Docker network cho communication giữa containers

8. Redeploy Khi Push Code Mới

Tự động (GitHub Actions)

1. push code lên branch main
2. GitHub Actions tự động:
 - a. Install dependencies
 - b. Type check (tsc)
 - c. Build Docker image (multi-stage)
 - d. Push image lên ghcr.io
 - e. SSH vào VPS
 - f. Pull image mới
 - g. Tag image cũ là `previous` (để rollback)
 - h. Chạy migration nếu `RUN_MIGRATIONS=true`
 - i. `docker compose up -d` (recreate app)
 - j. Health check (30s timeout)
 - k. Cleanup old images
3. App live với code mới!

Thủ công

```
# Trên VPS:
cd /opt/reiseblog

# Pull + deploy với tag cụ thể:
chmod +x scripts/deploy.sh
./scripts/deploy.sh abc1234

# Hoặc latest:
./scripts/deploy.sh latest
```

Khi có database schema changes:

```
# Trước khi deploy, set migration flag:
nano .env.production
# Đổi: RUN_MIGRATIONS=true

# Rồi deploy như thường (Actions sẽ chạy migration)
# Hoặc chạy thủ công:
chmod +x scripts/migrate.sh
./scripts/migrate.sh
```

9. Rollback

Rollback nhanh về version trước

```
cd /opt/reiseblog
chmod +x scripts/rollback.sh
./scripts/rollback.sh
```

Script sẽ:

1. Tìm image tag `previous` (được tự động lưu mỗi lần deploy)
2. Hỏi xác nhận
3. Khởi động lại với image cũ
4. Health check

Rollback về tag cụ thể

```
# Xem các image đang có:
docker images | grep reiseblog

# Deploy với tag cụ thể:
./scripts/deploy.sh abc1234
```

Rollback database (cẩn thận)

```
# Backup trước khi làm gì:
docker exec reiseblog-db pg_dump -U reiseblog reiseblog > backup_$(date +
%Y%m%d_%H%M%S).sql

# Restore:
cat backup_file.sql | docker exec -i reiseblog-db psql -U reiseblog -d reiseblog
```

10. Monitoring & Logs

Container status

```
# Tổng quan:
docker compose -f /opt/reiseblog/docker-compose.production.yml ps

# Chi tiết:
docker inspect reiseblog-app | grep -A5 Health
docker stats --no-stream
```

Application logs

```
# Next.js app:
docker logs reiseblog-app --tail 100
docker logs -f reiseblog-app          # follow real-time

# PostgreSQL:
docker logs reiseblog-db --tail 50

# Xem logs theo thời gian:
docker logs reiseblog-app --since 1h
docker logs reiseblog-app --since 2024-01-01T00:00:00
```

Nginx logs

```
# Access log:
sudo tail -f /var/log/nginx/reiseblog-access.log

# Error log:
sudo tail -f /var/log/nginx/reiseblog-error.log

# Xem request nhiều nhất:
sudo awk '{print $7}' /var/log/nginx/reiseblog-access.log | sort | uniq -c | sort -rn
| head 20

# Xem IP nhiều request:
sudo awk '{print $1}' /var/log/nginx/reiseblog-access.log | sort | uniq -c | sort -rn
| head 20
```

Health check nhanh

```
# Từ VPS:
curl -s -o /dev/null -w "HTTP %{http_code} - %{time_total}s\n" http://localhost:3000
curl -s -o /dev/null -w "HTTP %{http_code} - %{time_total}s\n" https://yourdomain.com

# SSL check:
curl -vI https://yourdomain.com 2>&1 | grep -E 'subject|expire|issuer'
```

Disk & Memory

```
# Disk usage:
df -h
docker system df

# Memory:
free -h
docker stats --no-stream

# Cleanup Docker:
docker system prune -f --volumes
```

11. Database Migrations

Chiến lược Migration an toàn

1. Khi nào chạy migration?

- Khi Prisma schema thay đổi (model mới, field mới, index mới)
- **TRƯỚC** khi deploy code mới sử dụng schema mới

2. Chạy ở đâu?

- Trong CI/CD pipeline (khuyến nghị)
- Hoặc thủ công trên VPS với `scripts/migrate.sh`

3. Flow an toàn:

- Backup database
- Chạy migration (`prisma migrate deploy`)
- Deploy code mới
- Health check
- Nếu lỗi → rollback code (migration không rollback tự động)

4. Backup trước migration:

```
bash
docker exec reiseblog-db pg_dump -U reiseblog reiseblog > \
  /opt/reiseblog/backups/pre-migration-$(date +%Y%m%d).sql
```

5. Không bao giờ:

- Chạy `prisma db push` trong production
- Chạy `prisma migrate reset` trong production
- Xóa migration files

12. Troubleshooting

App không khởi động

```
# Xem logs:
docker logs reiseblog-app --tail 100

# Kiểm tra env:
docker exec reiseblog-app env | grep -E 'DATABASE|NEXT|NODE'

# Kiểm tra DB connection:
docker exec reiseblog-app node -e
"const { PrismaClient } = require('@prisma/client'); new PrismaClient().\$connect().then(() => console.log('OK')).catch(e => console.error(e))"
```

502 Bad Gateway

```
# App có đang chạy không?
docker ps | grep reiseblog-app
curl http://localhost:3000

# Nginx config đúng không?
sudo nginx -t
sudo systemctl reload nginx
```

Database connection error

```
# PostgreSQL có healthy không?
docker inspect --format='{{.State.Health.Status}}' reiseblog-db

# Test connection:
docker exec -it reiseblog-db psql -U reiseblog -d reiseblog -c "SELECT 1"
```

SSL không hoạt động

```
# Kiểm tra cert:
sudo certbot certificates

# Renew thủ công:
sudo certbot renew --force-renewal
sudo systemctl reload nginx

# Kiểm tra SSL:
curl -vI https://yourdomain.com 2>&1 | grep -E 'SSL|subject|expire'
```

Docker disk full

```
# Xem disk usage:
docker system df

# Cleanup:
docker system prune -af --volumes
docker image prune -af
```

File Structure Tổng Kết

```

project/
├── .github/
│   └── workflows/
│       └── deploy.yml                # CI/CD pipeline
├── deploy/
│   ├── nginx/
│   │   ├── reiseblog.conf           # Nginx HTTPS config
│   │   └── nginx-initial-http.conf  # Nginx HTTP (tạm cho Certbot)
│   ├── scripts/
│   │   ├── deploy.sh               # Manual deploy
│   │   ├── rollback.sh             # Rollback
│   │   └── migrate.sh              # Manual migration
│   └── DEPLOY_GUIDE.md              # Tài liệu này
├── docker-compose.production.yml    # Docker Compose production
├── .env.production.example          # Template env
├── nextjs_space/
│   ├── Dockerfile                  # Multi-stage build
│   ├── deploy/
│   │   └── entrypoint.sh            # Container entrypoint
│   ├── prisma/
│   │   └── schema.prisma
│   └── ... (Next.js source)

```

Checklist Deploy Lần Đầu

- [] VPS đã có IP public
- [] Domain đã trỏ về VPS (A record)
- [] Docker + Docker Compose đã cài
- [] Nginx đã cài
- [] Certbot đã cài
- [] User deploy đã tạo (không dùng root)
- [] SSH key cho GitHub Actions đã setup
- [] GitHub Secrets đã cấu hình (SSH_HOST, SSH_USER, SSH_PORT, SSH_PRIVATE_KEY)
- [] VPS đã login ghcr.io
- [] UFW đã bật (chỉ port 22/80/443)
- [] .env.production đã cấu hình trên VPS
- [] docker-compose.production.yml đã copy lên VPS
- [] Nginx config đã cấu hình
- [] SSL đã cấp
- [] App healthy sau deploy
- [] Admin login hoạt động
- [] Fail2ban đã cài (optional)